18. L. J. Guibas and R. Sedgewick, "A Dichromatic Framework for Balanced Trees," *Proceedings of the Nineteenth Annual IEEE Symposium on Foundations of Computer Science* (1978), 8–21.
19. T. H. Hibbard, "Some Combinatorial Properties of Certain Trees with Applications to Searching and Sorting," *Journal of the ACM,* 9 (1962), 13–28.
20. A. T. Jonassen and D. E. Knuth, "A Trivial Algorithm Whose Analysis Isn't," *Journal of Computer and System Sciences,* 16 (1978), 301–322.
21. P. L. Karlton, S. H. Fuller, R. E. Scroggs, and E. B. Kaehler, "Performance of Height Balanced Trees," *Communications of the ACM,* 19 (1976), 23–28.
22. D. E. Knuth, *The Art of Computer Programming: Vol. 1: Fundamental Algorithms,* 3d ed., Addison-Wesley, Reading, Mass., 1997.
23. D. E. Knuth, *The Art of Computer Programming: Vol. 3: Sorting and Searching,* 2d ed., Addison-Wesley, Reading, Mass., 1998.
24. K. Melhorn, "A Partial Analysis of Height-Balanced Trees under Random Insertions and Deletions," *SIAM Journal of Computing,* 11 (1982), 748–760.
25. K. Melhorn, *Data Structures and Algorithms 1: Sorting and Searching,* Springer-Verlag, Berlin, 1984.
26. J. Nievergelt and E. M. Reingold, "Binary Search Trees of Bounded Balance," *SIAM Journal on Computing,* 2 (1973), 33–43.
27. A. J. Perlis and C. Thornton, "Symbol Manipulation in Threaded Lists," *Communications of the ACM,* 3 (1960), 195–204.
28. S. Sen and R. E. Tarjan, "Deletion Without Rebalancing in Balanced Binary Trees," *Proceedings of the Twentieth Symposium on Discrete Algorithms* (2010), 1490–1499.
29. D. D. Sleator and R. E. Tarjan, "Self-adjusting Binary Search Trees," *Journal of the ACM,* 32 (1985), 652–686.
30. D. D. Sleator, R. E. Tarjan, and W. P. Thurston, "Rotation Distance, Triangulations, and Hyperbolic Geometry," *Journal of the AMS* (1988), 647–682.
31. R. E. Tarjan, "Sequential Access in Splay Trees Takes Linear Time," *Combinatorica,* 5 (1985), 367–378.
32. A. C. Yao, "On Random 2–3 Trees," *Acta Informatica,* 9 (1978), 159–170.

This page intentionally left blank

CHAPTER 5

Hashing

In Chapter 4 we discussed the search tree ADT, which allowed various operations on a set of elements. In this chapter, we discuss the *hash table* ADT, which supports only a subset of the operations allowed by binary search trees.

The implementation of hash tables is frequently called **hashing**. Hashing is a technique used for performing insertions, deletions, and finds in constant average time. Tree operations that require any ordering information among the elements are not supported efficiently. Thus, operations such as `findMin`, `findMax`, and the printing of the entire table in sorted order in linear time are not supported.

The central data structure in this chapter is the **hash table**. We will . . .

- See several methods of implementing the hash table.
- Compare these methods analytically.
- Show numerous applications of hashing.
- Compare hash tables with binary search trees.

5.1 General Idea

The ideal hash table data structure is merely an array of some fixed size containing the items. As discussed in Chapter 4, generally a search is performed on some part (that is, data member) of the item. This is called the **key.** For instance, an item could consist of a string (that serves as the key) and additional data members (for instance, a name that is part of a large employee structure). We will refer to the table size as *TableSize,* with the understanding that this is part of a hash data structure and not merely some variable floating around globally. The common convention is to have the table run from 0 to *TableSize* − 1; we will see why shortly.

Each key is mapped into some number in the range 0 to *TableSize* − 1 and placed in the appropriate cell. The mapping is called a **hash function,** which ideally should be simple to compute and should ensure that any two distinct keys get different cells. Since there are a finite number of cells and a virtually inexhaustible supply of keys, this is clearly impossible, and thus we seek a hash function that distributes the keys evenly among the cells. Figure 5.1 is typical of a perfect situation. In this example, *john* hashes to 3, *phil* hashes to 4, *dave* hashes to 6, and *mary* hashes to 7.

0	
1	
2	
3	john 25000
4	phil 31250
5	
6	dave 27500
7	mary 28200
8	
9	

Figure 5.1 An ideal hash table

This is the basic idea of hashing. The only remaining problems deal with choosing a function, deciding what to do when two keys hash to the same value (this is known as a **collision**), and deciding on the table size.

5.2 Hash Function

If the input keys are integers, then simply returning *Key* mod *TableSize* is generally a reasonable strategy, unless *Key* happens to have some undesirable properties. In this case, the choice of hash function needs to be carefully considered. For instance, if the table size is 10 and the keys all end in zero, then the standard hash function is a bad choice. For reasons we shall see later, and to avoid situations like the one above, it is often a good idea to ensure that the table size is prime. When the input keys are random integers, then this function is not only very simple to compute but also distributes the keys evenly.

Usually, the keys are strings; in this case, the hash function needs to be chosen carefully.

One option is to add up the ASCII values of the characters in the string. The routine in Figure 5.2 implements this strategy.

The hash function depicted in Figure 5.2 is simple to implement and computes an answer quickly. However, if the table size is large, the function does not distribute the keys well. For instance, suppose that *TableSize* = 10,007 (10,007 is a prime number). Suppose all the keys are eight or fewer characters long. Since an ASCII character has an integer value that is always at most 127, the hash function typically can only assume values between 0 and 1,016, which is $127 * 8$. This is clearly not an equitable distribution!

Another hash function is shown in Figure 5.3. This hash function assumes that *Key* has at least three characters. The value 27 represents the number of letters in the English alphabet, plus the blank, and 729 is 27^2. This function examines only the first three characters, but if these are random and the table size is 10,007, as before, then we would expect a

```
int hash( const string & key, int tableSize )
{
    int hashVal = 0;

    for( char ch : key )
        hashVal += ch;

    return hashVal % tableSize;
}
```

Figure 5.2 A simple hash function

```
int hash( const string & key, int tableSize )
{
    return ( key[ 0 ] + 27 * key[ 1 ] + 729 * key[ 2 ] ) % tableSize;
}
```

Figure 5.3 Another possible hash function—not too good

reasonably equitable distribution. Unfortunately, English is not random. Although there are $26^3 = 17{,}576$ possible combinations of three characters (ignoring blanks), a check of a reasonably large online dictionary reveals that the number of different combinations is actually only 2,851. Even if none of *these* combinations collide, only 28 percent of the table can actually be hashed to. Thus this function, although easily computable, is also not appropriate if the hash table is reasonably large.

Figure 5.4 shows a third attempt at a hash function. This hash function involves all characters in the key and can generally be expected to distribute well (it computes $\sum_{i=0}^{KeySize-1} Key[KeySize - i - 1] \cdot 37^i$ and brings the result into proper range). The code computes a polynomial function (of 37) by use of Horner's rule. For instance, another way of computing $h_k = k_0 + 37k_1 + 37^2k_2$ is by the formula $h_k = ((k_2) * 37 + k_1) * 37 + k_0$. Horner's rule extends this to an nth degree polynomial.

```
/**
 * A hash routine for string objects.
 */
unsigned int hash( const string & key, int tableSize )
{
    unsigned int hashVal = 0;

    for( char ch : key )
        hashVal = 37 * hashVal + ch;

    return hashVal % tableSize;
}
```

Figure 5.4 A good hash function

The hash function takes advantage of the fact that overflow is allowed and uses `unsigned int` to avoid introducing a negative number.

The hash function described in Figure 5.4 is not necessarily the best with respect to table distribution, but it does have the merit of extreme simplicity and is reasonably fast. If the keys are very long, the hash function will take too long to compute. A common practice in this case is not to use all the characters. The length and properties of the keys would then influence the choice. For instance, the keys could be a complete street address. The hash function might include a couple of characters from the street address and perhaps a couple of characters from the city name and ZIP code. Some programmers implement their hash function by using only the characters in the odd spaces, with the idea that the time saved computing the hash function will make up for a slightly less evenly distributed function.

The main programming detail left is collision resolution. If, when an element is inserted, it hashes to the same value as an already inserted element, then we have a collision and need to resolve it. There are several methods for dealing with this. We will discuss two of the simplest: separate chaining and open addressing; then we will look at some more recently discovered alternatives.

5.3 Separate Chaining

The first strategy, commonly known as **separate chaining,** is to keep a list of all elements that hash to the same value. We can use the Standard Library list implementation. If space is tight, it might be preferable to avoid their use (since these lists are doubly linked and waste space). We assume for this section that the keys are the first 10 perfect squares and that the hashing function is simply $hash(x) = x \bmod 10$. (The table size is not prime but is used here for simplicity.) Figure 5.5 shows the resulting separate chaining hash table.

To perform a `search`, we use the hash function to determine which list to traverse. We then search the appropriate list. To perform an `insert`, we check the appropriate list to see whether the element is already in place (if duplicates are expected, an extra data member is

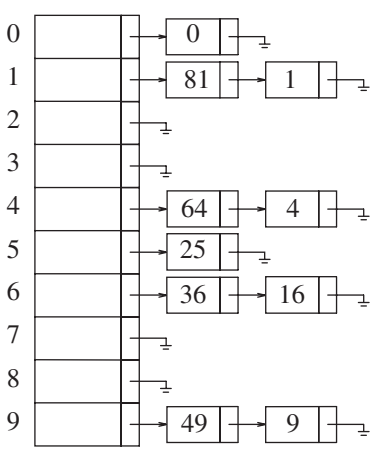

Figure 5.5 A separate chaining hash table

```
template <typename HashedObj>
class HashTable
{
  public:
    explicit HashTable( int size = 101 );

    bool contains( const HashedObj & x ) const;

    void makeEmpty( );
    bool insert( const HashedObj & x );
    bool insert( HashedObj && x );
    bool remove( const HashedObj & x );

  private:
    vector<list<HashedObj>> theLists;     // The array of Lists
    int currentSize;

    void rehash( );
    size_t myhash( const HashedObj & x ) const;
};
```

Figure 5.6 Type declaration for separate chaining hash table

usually kept, and this data member would be incremented in the event of a match). If the element turns out to be new, it can be inserted at the front of the list, since it is convenient and also because frequently it happens that recently inserted elements are the most likely to be accessed in the near future.

The class interface for a separate chaining implementation is shown in Figure 5.6. The hash table stores an array of linked lists, which are allocated in the constructor.

The class interface illustrates a syntax point: Prior to C++11, in the declaration of `theLists`, a space was required between the two >s; since >> is a C++ token, and because it is longer than >, >> would be recognized as the token. In C++11, this is no longer the case.

Just as the binary search tree works only for objects that are `Comparable`, the hash tables in this chapter work only for objects that provide a hash function and equality operators (`operator==` or `operator!=`, or possibly both).

Instead of requiring hash functions that take both the object and the table size as parameters, we have our hash functions take only the object as the parameter and return an appropriate integral type. The standard mechanism for doing this uses function objects, and the protocol for hash tables was introduced in C++11. Specifically, in C++11, hash functions can be expressed by the function object template:

```
template <typename Key>
class hash
{
  public:
    size_t operator() ( const Key & k ) const;
};
```

Default implementations of this template are provided for standard types such as `int` and `string`; thus, the hash function described in Figure 5.4 could be implemented as

```
template <>
class hash<string>
{
  public:
    size_t operator()( const string & key )
    {
        size_t hashVal = 0;

        for( char ch : key )
            hashVal = 37 * hashVal + ch;

        return hashVal;
    }
};
```

The type `size_t` is an unsigned integral type that represents the size of an object; therefore, it is guaranteed to be able to store an array index. A class that implements a hash table algorithm can then use calls to the generic hash function object to generate an integral type `size_t` and then scale the result into a suitable array index. In our hash tables, this is manifested in private member function `myhash`, shown in Figure 5.7.

Figure 5.8 illustrates an `Employee` class that can be stored in the generic hash table, using the `name` member as the key. The `Employee` class implements the `HashedObj` requirements by providing equality operators and a hash function object.

The code to implement `makeEmpty`, `contains`, and `remove` is shown in Figure 5.9.

Next comes the insertion routine. If the item to be inserted is already present, then we do nothing; otherwise, we place it in the list (see Fig. 5.10). The element can be placed anywhere in the list; using `push_back` is most convenient in our case. `whichList` is a reference variable; see Section 1.5.2 for a discussion of this use of reference variables.

Any scheme could be used besides linked lists to resolve the collisions; a binary search tree or even another hash table would work, but we expect that if the table is large and the hash function is good, all the lists should be short, so basic separate chaining makes no attempt to try anything complicated.

We define the load factor, λ, of a hash table to be the ratio of the number of elements in the hash table to the table size. In the example above, $\lambda = 1.0$. The average length of a list is λ. The effort required to perform a search is the constant time required to evaluate the hash function plus the time to traverse the list. In an unsuccessful search, the number

```
size_t myhash( const HashedObj & x ) const
{
    static hash<HashedObj> hf;
    return hf( x ) % theLists.size( );
}
```

Figure 5.7 `myHash` member function for hash tables

```
// Example of an Employee class
class Employee
{
  public:
    const string & getName( ) const
      { return name; }

    bool operator==( const Employee & rhs ) const
      { return getName( ) == rhs.getName( ); }
    bool operator!=( const Employee & rhs ) const
      { return !( *this == rhs; }

      // Additional public members not shown

  private:
    string name;
    double salary;
    int    seniority;

      // Additional private members not shown
};

template<>
class hash<Employee>
{
  public:
    size_t operator()( const Employee & item )
    {
        static hash<string> hf;
        return hf( item.getName( ) );
    }
};
```

Figure 5.8 Example of a class that can be used as a `HashedObj`

of nodes to examine is λ on average. A successful search requires that about $1 + (\lambda/2)$ links be traversed. To see this, notice that the list that is being searched contains the one node that stores the match plus zero or more other nodes. The expected number of "other nodes" in a table of N elements and M lists is $(N-1)/M = \lambda - 1/M$, which is essentially λ, since M is presumed large. On average, half the "other nodes" are searched, so combined with the matching node, we obtain an average search cost of $1 + \lambda/2$ nodes. This analysis shows that the table size is not really important but the load factor is. The general rule for separate chaining hashing is to make the table size about as large as the number of elements expected (in other words, let $\lambda \approx 1$). In the code in Figure 5.10, if the load factor exceeds 1, we expand the table size by calling `rehash` at line 10. `rehash` is discussed in Section 5.5. It is also a good idea, as mentioned before, to keep the table size prime to ensure a good distribution.

```
void makeEmpty( )
{
    for( auto & thisList : theLists )
        thisList.clear( );
}

bool contains( const HashedObj & x ) const
{
    auto & whichList = theLists[ myhash( x ) ];
    return find( begin( whichList ), end( whichList ), x ) != end( whichList );
}

bool remove( const HashedObj & x )
{
    auto & whichList = theLists[ myhash( x ) ];
    auto itr = find( begin( whichList ), end( whichList ), x );

    if( itr == end( whichList ) )
        return false;

    whichList.erase( itr );
    --currentSize;
    return true;
}
```

Figure 5.9 `makeEmpty`, `contains`, and `remove` routines for separate chaining hash table

```
bool insert( const HashedObj & x )
{
    auto & whichList = theLists[ myhash( x ) ];
    if( find( begin( whichList ), end( whichList ), x ) != end( whichList ) )
        return false;
    whichList.push_back( x );

        // Rehash; see Section 5.5
    if( ++currentSize > theLists.size( ) )
        rehash( );

    return true;
}
```

Figure 5.10 `insert` routine for separate chaining hash table

5.4 Hash Tables without Linked Lists

Separate chaining hashing has the disadvantage of using linked lists. This could slow the algorithm down a bit because of the time required to allocate new cells (especially in other languages) and essentially requires the implementation of a second data structure. An alternative to resolving collisions with linked lists is to try alternative cells until an empty cell is found. More formally, cells $h_0(x), h_1(x), h_2(x), \ldots$ are tried in succession, where $h_i(x) = (hash(x) + f(i)) \mod TableSize$, with $f(0) = 0$. The function, f, is the collision resolution strategy. Because all the data go inside the table, a bigger table is needed in such a scheme than for separate chaining hashing. Generally, the load factor should be below $\lambda = 0.5$ for a hash table that doesn't use separate chaining. We call such tables **probing hash tables**. We now look at three common collision resolution strategies.

5.4.1 Linear Probing

In linear probing, f is a linear function of i, typically $f(i) = i$. This amounts to trying cells sequentially (with wraparound) in search of an empty cell. Figure 5.11 shows the result of inserting keys {89, 18, 49, 58, 69} into a hash table using the same hash function as before and the collision resolution strategy, $f(i) = i$.

The first collision occurs when 49 is inserted; it is put in the next available spot, namely, spot 0, which is open. The key 58 collides with 18, 89, and then 49 before an empty cell is found three away. The collision for 69 is handled in a similar manner. As long as the table is big enough, a free cell can always be found, but the time to do so can get quite large. Worse, even if the table is relatively empty, blocks of occupied cells start forming. This effect, known as **primary clustering,** means that any key that hashes into the cluster will require several attempts to resolve the collision, and then it will add to the cluster.

Although we will not perform the calculations here, it can be shown that the expected number of probes using linear probing is roughly $\frac{1}{2}(1 + 1/(1 - \lambda)^2)$ for insertions and

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1					58	58
2						69
3						
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

Figure 5.11 Hash table with linear probing, after each insertion

unsuccessful searches, and $\frac{1}{2}(1 + 1/(1 - \lambda))$ for successful searches. The calculations are somewhat involved. It is easy to see from the code that insertions and unsuccessful searches require the same number of probes. A moment's thought suggests that, on average, successful searches should take less time than unsuccessful searches.

The corresponding formulas, if clustering is not a problem, are fairly easy to derive. We will assume a very large table and that each probe is independent of the previous probes. These assumptions are satisfied by a *random* collision resolution strategy and are reasonable unless λ is very close to 1. First, we derive the expected number of probes in an unsuccessful search. This is just the expected number of probes until we find an empty cell. Since the fraction of empty cells is $1 - \lambda$, the number of cells we expect to probe is $1/(1 - \lambda)$. The number of probes for a successful search is equal to the number of probes required when the particular element was inserted. When an element is inserted, it is done as a result of an unsuccessful search. Thus, we can use the cost of an unsuccessful search to compute the average cost of a successful search.

The caveat is that λ changes from 0 to its current value, so that earlier insertions are cheaper and should bring the average down. For instance, in the table in Figure 5.11, $\lambda = 0.5$, but the cost of accessing 18 is determined when 18 is inserted. At that point, $\lambda = 0.2$. Since 18 was inserted into a relatively empty table, accessing it should be easier than accessing a recently inserted element, such as 69. We can estimate the average by using an integral to calculate the mean value of the insertion time, obtaining

$$I(\lambda) = \frac{1}{\lambda}\int_0^{\lambda} \frac{1}{1-x}dx = \frac{1}{\lambda}\ln\frac{1}{1-\lambda}$$

These formulas are clearly better than the corresponding formulas for linear probing. Clustering is not only a theoretical problem but actually occurs in real implementations.

Figure 5.12 compares the performance of linear probing (dashed curves) with what would be expected from more random collision resolution. Successful searches are indicated by an *S,* and unsuccessful searches and insertions are marked with *U* and *I,* respectively.

If $\lambda = 0.75$, then the formula above indicates that 8.5 probes are expected for an insertion in linear probing. If $\lambda = 0.9$, then 50 probes are expected, which is unreasonable. This compares with 4 and 10 probes for the respective load factors if clustering were not a problem. We see from these formulas that linear probing can be a bad idea if the table is expected to be more than half full. If $\lambda = 0.5$, however, only 2.5 probes are required on average for insertion, and only 1.5 probes are required, on average, for a successful search.

5.4.2 Quadratic Probing

Quadratic probing is a collision resolution method that eliminates the primary clustering problem of linear probing. Quadratic probing is what you would expect—the collision function is quadratic. The popular choice is $f(i) = i^2$. Figure 5.13 shows the resulting hash table with this collision function on the same input used in the linear probing example.

When 49 collides with 89, the next position attempted is one cell away. This cell is empty, so 49 is placed there. Next, 58 collides at position 8. Then the cell one away is

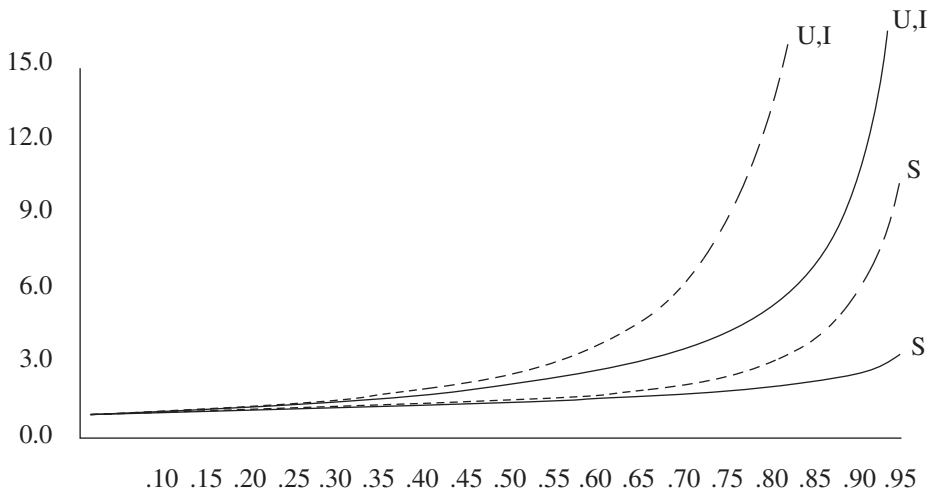

Figure 5.12 Number of probes plotted against load factor for linear probing (dashed) and random strategy (*S* is successful search, *U* is unsuccessful search, and *I* is insertion)

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1						
2					58	58
3						69
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

Figure 5.13 Hash table with quadratic probing, after each insertion

tried, but another collision occurs. A vacant cell is found at the next cell tried, which is $2^2 = 4$ away. 58 is thus placed in cell 2. The same thing happens for 69.

For linear probing, it is a bad idea to let the hash table get nearly full, because performance degrades. For quadratic probing, the situation is even more drastic: There is no guarantee of finding an empty cell once the table gets more than half full, or even before the table gets half full if the table size is not prime. This is because at most half of the table can be used as alternative locations to resolve collisions.

Indeed, we prove now that if the table is half empty and the table size is prime, then we are always guaranteed to be able to insert a new element.

Theorem 5.1

If quadratic probing is used, and the table size is prime, then a new element can always be inserted if the table is at least half empty.

Proof

Let the table size, *TableSize,* be an (odd) prime greater than 3. We show that the first $\lceil TableSize/2 \rceil$ alternative locations (including the initial location $h_0(x)$) are all distinct. Two of these locations are $h(x) + i^2 \pmod{TableSize}$ and $h(x) + j^2 \pmod{TableSize}$, where $0 \le i, j \le \lfloor TableSize/2 \rfloor$. Suppose, for the sake of contradiction, that these locations are the same, but $i \neq j$. Then

$$\begin{aligned} h(x) + i^2 &= h(x) + j^2 && \pmod{TableSize} \\ i^2 &= j^2 && \pmod{TableSize} \\ i^2 - j^2 &= 0 && \pmod{TableSize} \\ (i - j)(i + j) &= 0 && \pmod{TableSize} \end{aligned}$$

Since *TableSize* is prime, it follows that either $(i - j)$ or $(i + j)$ is equal to 0 (mod *TableSize*). Since i and j are distinct, the first option is not possible. Since $0 \le i, j \le \lfloor TableSize/2 \rfloor$, the second option is also impossible. Thus, the first $\lceil TableSize/2 \rceil$ alternative locations are distinct. If at most $\lfloor TableSize/2 \rfloor$ positions are taken, then an empty spot can always be found.

If the table is even one more than half full, the insertion could fail (although this is extremely unlikely). Therefore, it is important to keep this in mind. It is also crucial that the table size be prime.[1] If the table size is not prime, the number of alternative locations can be severely reduced. As an example, if the table size were 16, then the only alternative locations would be at distances 1, 4, or 9 away.

Standard deletion cannot be performed in a probing hash table, because the cell might have caused a collision to go past it. For instance, if we remove 89, then virtually all the remaining `find` operations will fail. Thus, probing hash tables require lazy deletion, although in this case there really is no laziness implied.

The class interface required to implement probing hash tables is shown in Figure 5.14. Instead of an array of lists, we have an array of hash table entry cells. The nested class `HashEntry` stores the state of an entry in the `info` member; this state is either `ACTIVE`, `EMPTY`, or `DELETED`.

We use a standard enumerated type.

```
enum EntryType { ACTIVE, EMPTY, DELETED };
```

Constructing the table (Fig. 5.15) consists of setting the `info` member to `EMPTY` for each cell. `contains(x)`, shown in Figure 5.16, invokes private member functions `isActive` and `findPos`. The `private` member function `findPos` performs the collision resolution. We ensure in the `insert` routine that the hash table is at least twice as large as the number of elements in the table, so quadratic resolution will always work. In the implementation

[1] If the table size is a prime of the form $4k + 3$, and the quadratic collision resolution strategy $f(i) = \pm i^2$ is used, then the entire table can be probed. The cost is a slightly more complicated routine.

```
template <typename HashedObj>
class HashTable
{
  public:
    explicit HashTable( int size = 101 );

    bool contains( const HashedObj & x ) const;

    void makeEmpty( );
    bool insert( const HashedObj & x );
    bool insert( HashedObj && x );
    bool remove( const HashedObj & x );

    enum EntryType { ACTIVE, EMPTY, DELETED };

  private:
    struct HashEntry
    {
        HashedObj element;
        EntryType info;

        HashEntry( const HashedObj & e = HashedObj{ }, EntryType i = EMPTY )
          : element{ e }, info{ i } { }
        HashEntry( HashedObj && e, EntryType i = EMPTY )
          : element{ std::move( e ) }, info{ i } { }
    };

    vector<HashEntry> array;
    int currentSize;

    bool isActive( int currentPos ) const;
    int findPos( const HashedObj & x ) const;
    void rehash( );
    size_t myhash( const HashedObj & x ) const;
};
```

Figure 5.14 Class interface for hash tables using probing strategies, including the nested `HashEntry` class

in Figure 5.16, elements that are marked as deleted count as being in the table. This can cause problems, because the table can get too full prematurely. We shall discuss this item presently.

Lines 12 to 15 represent the fast way of doing quadratic resolution. From the definition of the quadratic resolution function, $f(i) = f(i-1) + 2i - 1$, so the next cell to try is a distance from the previous cell tried and this distance increases by 2 on successive probes.

```
    explicit HashTable( int size = 101 ) : array( nextPrime( size ) )
      { makeEmpty( ); }

    void makeEmpty( )
    {
        currentSize = 0;
        for( auto & entry : array )
            entry.info = EMPTY;
    }
```

Figure 5.15 Routines to initialize quadratic probing hash table

```
1   bool contains( const HashedObj & x ) const
2     { return isActive( findPos( x ) ); }
3
4   int findPos( const HashedObj & x ) const
5   {
6       int offset = 1;
7       int currentPos = myhash( x );
8
9       while( array[ currentPos ].info != EMPTY &&
10             array[ currentPos ].element != x )
11      {
12          currentPos += offset;  // Compute ith probe
13          offset += 2;
14          if( currentPos >= array.size( ) )
15              currentPos -= array.size( );
16      }
17
18      return currentPos;
19  }
20
21  bool isActive( int currentPos ) const
22    { return array[ currentPos ].info == ACTIVE; }
```

Figure 5.16 `contains` routine (and `private` helpers) for hashing with quadratic probing

If the new location is past the array, it can be put back in range by subtracting *TableSize*. This is faster than the obvious method, because it avoids the multiplication and division that seem to be required. An important warning: The order of testing at lines 9 and 10 is important. Don't switch it!

The final routine is insertion. As with separate chaining hashing, we do nothing if `x` is already present. It is a simple modification to do something else. Otherwise, we place it at the spot suggested by the `findPos` routine. The code is shown in Figure 5.17. If the load

```
bool insert( const HashedObj & x )
{
        // Insert x as active
    int currentPos = findPos( x );
    if( isActive( currentPos ) )
        return false;

    array[ currentPos ].element = x;
    array[ currentPos ].info = ACTIVE;

        // Rehash; see Section 5.5
    if( ++currentSize > array.size( ) / 2 )
        rehash( );

    return true;
}

bool remove( const HashedObj & x )
{
    int currentPos = findPos( x );
    if( !isActive( currentPos ) )
        return false;

    array[ currentPos ].info = DELETED;
    return true;
}
```

Figure 5.17 Some `insert` and `remove` routines for hash tables with quadratic probing

factor exceeds 0.5, the table is full and we enlarge the hash table. This is called rehashing and is discussed in Section 5.5. Figure 5.17 also shows `remove`.

Although quadratic probing eliminates primary clustering, elements that hash to the same position will probe the same alternative cells. This is known as **secondary clustering**. Secondary clustering is a slight theoretical blemish. Simulation results suggest that it generally causes less than an extra half probe per search. The following technique eliminates this, but does so at the cost of computing an extra hash function.

5.4.3 Double Hashing

The last collision resolution method we will examine is **double hashing**. For double hashing, one popular choice is $f(i) = i \cdot hash_2(x)$. This formula says that we apply a second hash function to x and probe at a distance $hash_2(x), 2hash_2(x), \ldots$, and so on. A poor choice of $hash_2(x)$ would be disastrous. For instance, the obvious choice $hash_2(x) = x \bmod 9$ would not help if 99 were inserted into the input in the previous examples. Thus, the function must never evaluate to zero. It is also important to make sure all cells can be probed (this is not possible in the example below, because the table size is not prime). A function such

	Empty Table	After 89	After 18	After 49	After 58	After 69
0						69
1						
2						
3					58	58
4						
5						
6				49	49	49
7						
8			18	18	18	18
9		89	89	89	89	89

Figure 5.18 Hash table with double hashing, after each insertion

as $hash_2(x) = R - (x \bmod R)$, with R a prime smaller than *TableSize,* will work well. If we choose $R = 7$, then Figure 5.18 shows the results of inserting the same keys as before.

The first collision occurs when 49 is inserted. $hash_2(49) = 7 - 0 = 7$, so 49 is inserted in position 6. $hash_2(58) = 7 - 2 = 5$, so 58 is inserted at location 3. Finally, 69 collides and is inserted at a distance $hash_2(69) = 7 - 6 = 1$ away. If we tried to insert 60 in position 0, we would have a collision. Since $hash_2(60) = 7 - 4 = 3$, we would then try positions 3, 6, 9, and then 2 until an empty spot is found. It is generally possible to find some bad case, but there are not too many here.

As we have said before, the size of our sample hash table is not prime. We have done this for convenience in computing the hash function, but it is worth seeing why it is important to make sure the table size is prime when double hashing is used. If we attempt to insert 23 into the table, it would collide with 58. Since $hash_2(23) = 7 - 2 = 5$, and the table size is 10, we essentially have only one alternative location, and it is already taken. Thus, if the table size is not prime, it is possible to run out of alternative locations prematurely. However, if double hashing is correctly implemented, simulations imply that the expected number of probes is almost the same as for a random collision resolution strategy. This makes double hashing theoretically interesting. Quadratic probing, however, does not require the use of a second hash function and is thus likely to be simpler and faster in practice, especially for keys like strings whose hash functions are expensive to compute.

5.5 Rehashing

If the table gets too full, the running time for the operations will start taking too long, and insertions might fail for open addressing hashing with quadratic resolution. This can happen if there are too many removals intermixed with insertions. A solution, then, is to build another table that is about twice as big (with an associated new hash function) and scan down the entire original hash table, computing the new hash value for each (nondeleted) element and inserting it in the new table.

0	6
1	15
2	
3	24
4	
5	
6	13

Figure 5.19 Hash table with linear probing with input 13, 15, 6, 24

As an example, suppose the elements 13, 15, 24, and 6 are inserted into a linear probing hash table of size 7. The hash function is $h(x) = x \mod 7$. The resulting hash table appears in Figure 5.19.

If 23 is inserted into the table, the resulting table in Figure 5.20 will be over 70 percent full. Because the table is so full, a new table is created. The size of this table is 17, because this is the first prime that is twice as large as the old table size. The new hash function is then $h(x) = x \mod 17$. The old table is scanned, and elements 6, 15, 23, 24, and 13 are inserted into the new table. The resulting table appears in Figure 5.21.

This entire operation is called **rehashing**. This is obviously a very expensive operation; the running time is $O(N)$, since there are N elements to rehash and the table size is roughly $2N$, but it is actually not all that bad, because it happens very infrequently. In particular, there must have been $N/2$ insertions prior to the last rehash, so it essentially adds a constant cost to each insertion. This is why the new table is made twice as large as the old table. If this data structure is part of the program, the effect is not noticeable. On the other hand, if the hashing is performed as part of an interactive system, then the unfortunate user whose insertion caused a rehash could see a slowdown.

0	6
1	15
2	23
3	24
4	
5	
6	13

Figure 5.20 Hash table with linear probing after 23 is inserted

0	
1	
2	
3	
4	
5	
6	6
7	23
8	24
9	
10	
11	
12	
13	13
14	
15	15
16	

Figure 5.21 Hash table after rehashing

Rehashing can be implemented in several ways with quadratic probing. One alternative is to rehash as soon as the table is half full. The other extreme is to rehash only when an insertion fails. A third, middle-of-the-road strategy is to rehash when the table reaches a certain load factor. Since performance does degrade as the load factor increases, the third strategy, implemented with a good cutoff, could be best.

Rehashing for separate chaining hash tables is similar. Figure 5.22 shows that rehashing is simple to implement and provides an implementation for separate chaining rehashing.

5.6 Hash Tables in the Standard Library

In C++11, the Standard Library includes hash table implementations of sets and maps—namely, `unordered_set` and `unordered_map`, which parallel `set` and `map`. The items in the `ordered_set` (or the keys in the `unordered_map`) must provide an overloaded `operator==` and a `hash` function, as described earlier, in Section 5.3. Just as the `set` and `map` templates can